Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 3 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

Contents

1 Software Requirements Specification (SRS) Summary **2**
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 User Classes . . . 2

2 Functional Requirements **2**

3 Non-Functional Requirements **3**

4 Technology Stack **3**

5 Use Case Diagram **4**
5.1 Main Use Cases . . . 5

6 System Architecture **5**

7 Data Model **6**
7.1 Item Table . . . 6
7.2 Transaction Table . . . 7

8 Entity Relationship (ER) Diagram **7**

9 REST API List **7**

10 Local Development Setup **8**
10.1 Required Software . . . 8
10.2 Local Development Flow . . . 8
10.3 Planned Project Structure . . . 9
10.4 Initial Local Setup . . . 9

11 Conclusion **10**

1. Software Requirements Specification (SRS) Summary

1.1 Purpose

The purpose of the Automated Grocery Inventory Dashboard is to provide a simple web-based system for managing grocery inventory in a centralized manner. The system reduces the need for manual stock records and helps users monitor product availability, update stock, and identify products requiring replenishment.

The application is designed as a small Minimum Viable Product (MVP) and focuses on essential inventory operations along with search, stock status, and exception alerts.

1.2 Scope

The application will allow store staff or managers to:

- Add, update, delete, and view grocery items.
- Record stock-in and stock-out transactions.
- View current stock status.
- Search and filter inventory items.
- View low-stock and out-of-stock alerts.
- Monitor inventory information through a dashboard.

The project will later integrate DevOps practices using Git, GitHub, Jenkins, Selenium, Docker, and Ansible as defined in the 15-week project plan.

1.3 User Classes

- **Store Staff:** Manage products and record stock transactions.
- **Store Manager/Owner:** Monitor inventory, stock status, and alerts.
- **System Administrator:** Maintain the application and deployment environment.

2. Functional Requirements

The major functional requirements of the Grocery Inventory Dashboard are listed below.

Requirement ID	Description
FR-01	Add a new grocery item to the inventory.
FR-02	Update existing grocery item details.
FR-03	Delete an existing grocery item.
FR-04	View the complete inventory list.
FR-05	Record stock-in transactions when new stock is received.
FR-06	Record stock-out transactions when stock is sold or removed.
FR-07	Display the current stock status of each product.
FR-08	Search products by name and filter them by category.
FR-09	Display low-stock and out-of-stock alerts.
FR-10	Display basic inventory statistics on the dashboard.

3. Non-Functional Requirements

Requirement	Description
Performance	The system should provide quick responses for normal inventory operations.
Reliability	Stock quantities and transactions should be stored accurately.
Security	Inventory operations should be accessible only to authorized users if authentication is implemented.
Usability	The dashboard should have a simple and easy-to-understand interface.
Maintainability	The application should use a modular structure for easier updates.
Scalability	The system should allow additional products and categories to be added easily.
Availability	The application should remain accessible during normal project demonstrations.

4. Technology Stack

The selected technologies are lightweight, widely used, and suitable for implementing the application and its later DevOps workflow.

Component	Technology	Purpose
Frontend	HTML, CSS, Bootstrap, JavaScript	Provides the user interface.
Backend	Java + Spring Boot	Handles business logic and REST APIs.
Build Tool	Maven	Manages dependencies and builds the application.
Database	MySQL	Stores inventory and transaction data.
Deployment Target	Tomcat	Hosts the Spring Boot application for deployment.
Version Control	Git	Tracks source code changes.
Repository	GitHub	Stores and manages the project source code.
CI/CD	Jenkins	Automates build, testing, and deployment in later stages.
Testing	Selenium WebDriver	Automates browser-based testing.
Containerization	Docker	Provides a consistent deployment environment.
Configuration Management	Ansible	Automates server configuration and deployment.

5. Use Case Diagram

The use case diagram represents the main interactions between users and the Grocery Inventory Dashboard.

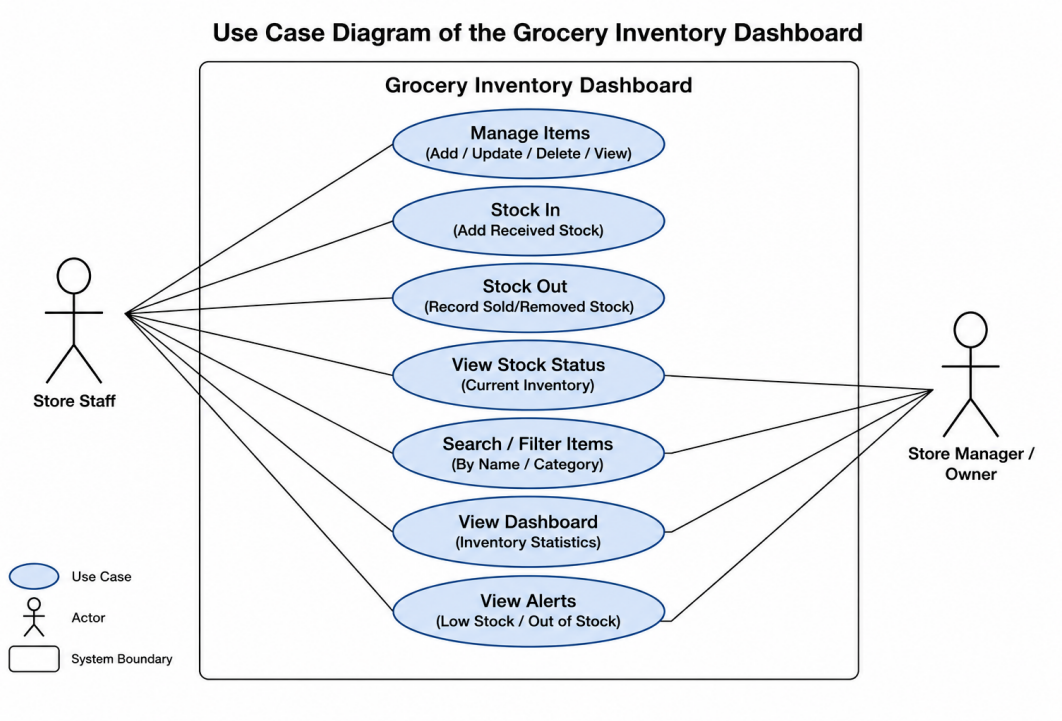


Figure 1: Use Case Diagram of the Grocery Inventory Dashboard

5.1 Main Use Cases

Use Case	Description
Manage Items	Add, update, delete, and view grocery items.
Stock In	Increase product quantity when new stock is received.
Stock Out	Decrease product quantity when stock is sold or removed.
View Stock Status	Check whether products are in stock, low in stock, or out of stock.
Search and Filter	Find products by name or category.
View Dashboard	View overall inventory statistics.
View Alerts	Identify low-stock and out-of-stock products.

6. System Architecture

The application follows a simple three-layer architecture consisting of the presentation layer, application layer, and database layer.

- **Presentation Layer:** Provides the web interface using HTML, CSS, Bootstrap, and JavaScript.
- **Application Layer:** Spring Boot handles REST APIs, business logic, inventory operations, and alert conditions.

- **Data Layer:** MySQL stores product details, stock transactions, and inventory information.

The application will use REST APIs for communication between the frontend and backend. The backend will validate requests, update the database, and return the required information to the frontend.

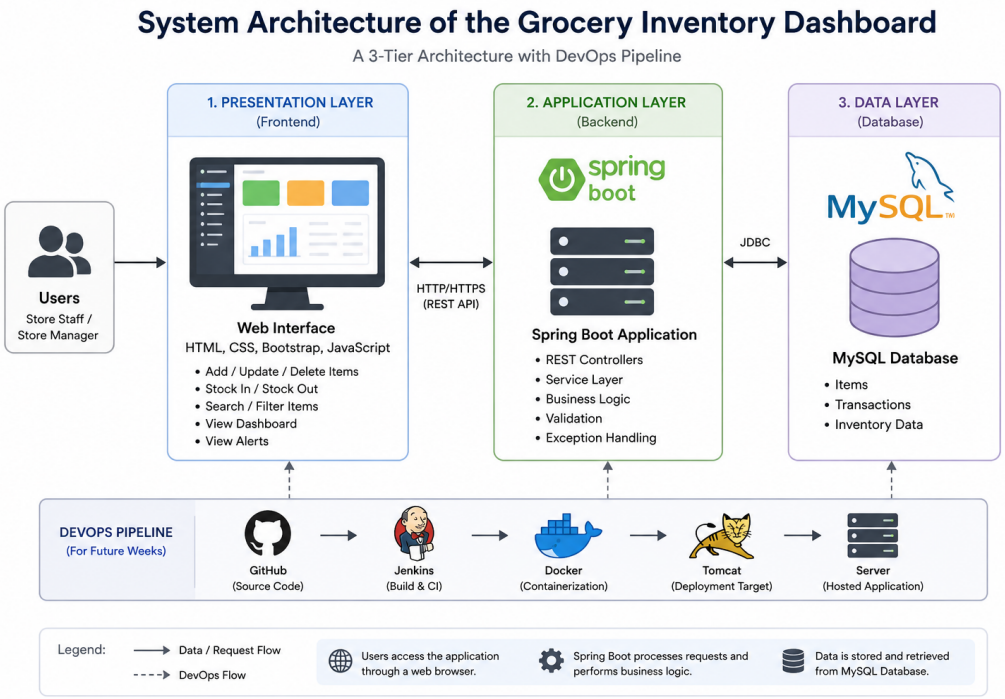


Figure 2: System Architecture of the Grocery Inventory Dashboard

7. Data Model

The MVP requires two main entities: **Item** and **Transaction**. The Item entity stores the current inventory information, while the Transaction entity records stock movement.

7.1 Item Table

Field	Data Type	Description
item_id	Integer	Unique identifier for the item.
item_name	String	Name of the grocery item.
category	String	Category of the item.
unit_price	Decimal	Price per unit.
quantity	Integer	Current available quantity.
reorder_level	Integer	Minimum quantity before a low-stock alert.

7.2 Transaction Table

Field	Data Type	Description
transaction_id	Integer	Unique identifier for the transaction.
item_id	Integer	Reference to the affected item.
transaction_type	String	Indicates Stock In or Stock Out.
quantity	Integer	Quantity involved in the transaction.
transaction_date	Date/Time	Date and time of the transaction.
status	String	Status of the transaction.

8. Entity Relationship (ER) Diagram

The Item entity can have multiple associated stock transactions. Each transaction belongs to one inventory item.

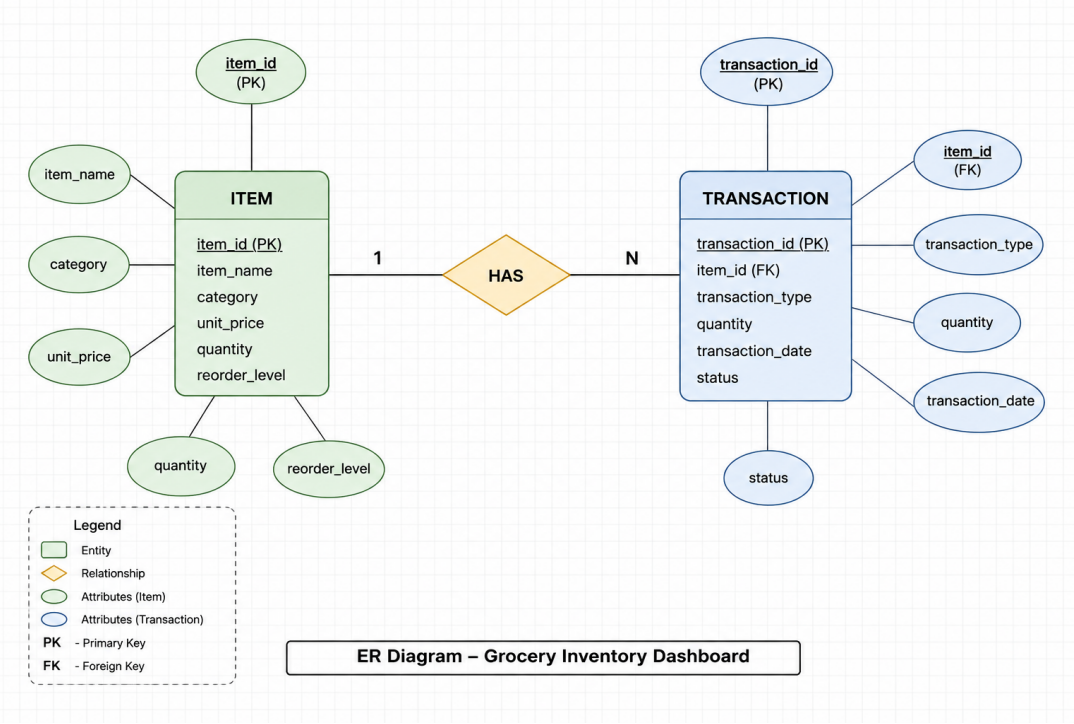


Figure 3: Entity Relationship Diagram

The relationship can be summarized as:

One Item ⟶ Many Transactions

9. REST API List

The backend will expose REST APIs for the main inventory operations.

Method	Endpoint	Description
GET	/api/items	View all inventory items.
POST	/api/items	Add a new grocery item.
GET	/api/items/{id}	View a specific item.
PUT	/api/items/{id}	Update item details.
DELETE	/api/items/{id}	Delete an item.
POST	/api/transactions	Create a stock transaction.
PUT	/api/transactions/{id}	Update a transaction if required.
GET	/api/transactions	View stock transactions.
GET	/api/items/search	Search or filter inventory items.
GET	/api/dashboard	Retrieve dashboard statistics.
GET	/api/alerts	Retrieve low-stock and out-of-stock alerts.

These APIs are planned for implementation during the development phase.

10. Local Development Setup

The application will initially be developed and tested on a local machine before being integrated with Jenkins, Docker, and Ansible in later weeks.

10.1 Required Software

- Java JDK 21
- Spring Boot
- Apache Maven
- MySQL Server
- Git
- GitHub Account
- Visual Studio Code or IntelliJ IDEA
- Google Chrome

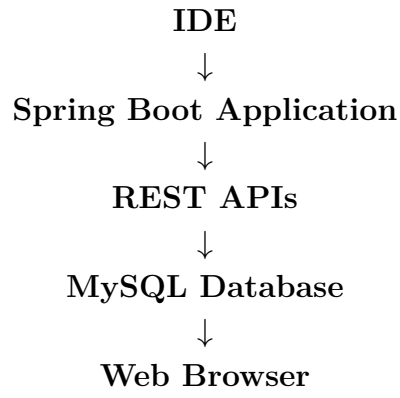
Tools such as Jenkins, Selenium, Docker, and Ansible will be introduced in their respective weeks according to the project timeline.

10.2 Local Development Flow

The initial development flow is:

Developer

↓



10.3 Planned Project Structure

```
automated-grocery-inventory-dashboard/
|
+-- src/
|   +-- main/
|   |   +-- java/
|   |   +-- resources/
|   |       +-- static/
|   |       +-- templates/
|   |
|   +-- test/
|
+-- docs/
+-- screenshots/
+-- pom.xml
+-- README.md
+-- .gitignore
```

10.4 Initial Local Setup

The basic setup process is:

1. Install Java JDK and verify the Java version.
2. Install Maven and verify the Maven version.
3. Install and configure MySQL.
4. Create the Spring Boot project using Maven.
5. Configure the MySQL database connection.
6. Run the application locally.
7. Open the application in a web browser and verify that it is accessible.

11. Conclusion

The Requirements, Architecture, and Technology Setup phase establishes the technical foundation for the Automated Grocery Inventory Dashboard. The selected Spring Boot, Maven, and MySQL stack provides a simple structure for developing the MVP and integrating DevOps tools in later weeks.

The defined requirements, use cases, architecture, data model, and API list provide a clear plan for implementation. The local setup will be completed before moving to GitHub and feature development in the upcoming weeks.